arXiv:2305.16349v1 [cs.CL] 24 May 2023

Lexinvariant Language Models

Qian Huang[1]
qhwang@cs.stanford.edu

Eric Zelikman[1]
ezelikman@cs.stanford.edu

Sarah Li Chen[1]
sachen@stanford.edu

Yuhuai Wu[12]
yuhuai@cs.stanford.edu

Gregory Valiant[1]
gvaliant@cs.stanford.edu

Percy Liang[1]
pliang@cs.stanford.edu

[1]Stanford University
[2]Google Research

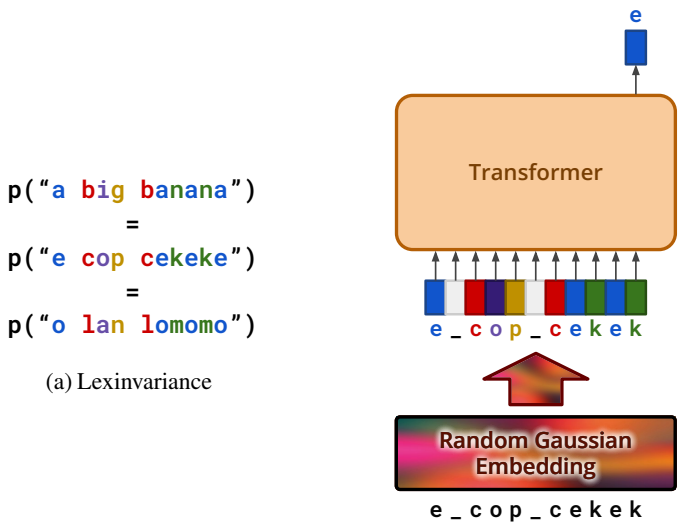
Abstract

Token embeddings, a mapping from discrete lexical symbols to continuous vectors, are at the heart of any language model (LM). However, lexical symbol meanings can also be determined and even redefined by their structural role in a long context. In this paper, we ask: is it possible for a language model to be performant without *any* fixed token embeddings? Such a language model would have to rely entirely on the co-occurence and repetition of tokens in the context rather than the *a priori* identity of any token. To answer this, we study *lexinvariant* language models that are invariant to lexical symbols and therefore do not need fixed token embeddings in practice. First, we prove that we can construct a lexinvariant LM to converge to the true language model at a uniform rate that is polynomial in terms of the context length, with a constant factor that is sublinear in the vocabulary size. Second, to build a lexinvariant LM, we simply encode tokens using random Gaussian vectors, such that each token maps to the same representation within each sequence but different representations across sequences. Empirically, we demonstrate that it can indeed attain perplexity comparable to that of a standard language model, given a sufficiently long context. We further explore two properties of the lexinvariant language models: First, given text generated from a substitution cipher of English, it implicitly implements Bayesian in-context deciphering and infers the mapping to the underlying real tokens with high accuracy. Second, it has on average 4X better accuracy over synthetic in-context reasoning tasks. Finally, we discuss regularizing standard language models towards lexinvariance and potential practical applications.

1 Introduction

All language processing systems rely on a stable lexicon, which assumes that a token (a word or subword such as *tree*) has a consistent contribution to the meaning of a text (though of course this meaning is mediated by context). In neural language models (LMs), this contribution is the token embedding, which *stably* maps each token into a continuous vector [21, 16, 17, 7, 6]. However, in real language, a token's contribution might be determined by its structural role; in math and code, novel variable names are arbitrarily defined to carry new meaning, and poems such as Jabberwocky exploit humans' lexical flexibility in interpreting novel words such as *vorpal*. Besides standard language understanding, this lexical flexibility also correlates with a stronger in-context reasoning performance. For example, GPT-3 [6] and other large language models that demonstrate high lexical flexibility show strong performance on tasks involving in-context reasoning over new concepts and rules.

Preprint. Under review.



(a) Lexinvariance

(b) Lexinvariant Language Model

Figure 1: Definition (a) and construction (b) of lexinvariant language model

Motivated by the above, we ask whether we can push this flexibility to the extreme: can we build a language model without *any* stable lexical mapping? To this end, we formulate and study such *lexinvariant* language models. We define a lexinvariant language model as a language model that assigns the same probability to all lexical permutations of a sequence. Formally, we define a lexical permutation π to be a one-to-one mapping of a set of lexical symbols [1] onto itself. Then the lexinvariant language model is defined as a language model over the symbol sequence $x_1,\ldots,x_n$ with the following property:

$$p(x_1,\ldots,x_n) = p(\pi(x_1),\ldots,\pi(x_n))\ \forall\pi \tag{1}$$

For example, a lexinvariant language model (whose vocabulary is letters and space) should assign the same probability to the phrase "`a big banana`" as "`e cop cekeke`" because the two are the same up to the permutation $\pi = \{\texttt{a:e},\texttt{b:c},\texttt{i:o},\texttt{n:k},\texttt{g:p},\cdots\}$ (Figure 1a).

The central question is: how well can lexinvariant language models predict the next token given an increasingly long context? We find the answer is almost as well as standard language models, both theoretically and empirically. This is rather surprising given that lexinvariance seems like a strong limitation (a model doesnt́ know what any individual symbol means!) However, the intuition is that given longer contexts, a lexinvariant model can both infer the latent permutation π (lazily) up to whatever ambiguity is present in the language model, and do the standard next word prediction task jointly.

Theoretically, we prove that a constructed lexinvariant language model can converge to the true language model as the context length increases—that is, the average L1 distance between the predictions of the two models decreases with a convergence rate of $O\left((\frac{d}{T})^{\frac{1}{4}}\right)$, where T is the length of the context and d is the vocabulary size, and where the big-O notation hides polylogarithmic factors of d and T and an absolute constant that is *independent* of the language model.

Empirically, we train a lexinvariant LM by replacing standard embeddings in a decoder-only Transformer [24] with per-sequence random Gaussian vectors, such that the same symbols get the same embedding within each sequence but get different embedding across sequences (Figure 1b). We indeed see that the perplexity gap between the lexinvariant LM and the standard LM shrinks as context length increases, as shown in Section 3.2. With a 150M parameters Transformer and a small character-level vocabulary (130 tokens), the average perplexity gap shrinks from 9X to less than 1X the average perplexity of a standard LM after observing 512 tokens over The Pile [9]. With a larger 32K vocabulary, the gap also shrinks, especially on the more structured text like GitHub code, albeit at a much slower rate.

We then explore two additional properties of the lexinvariant LM: in-context deciphering and symbol manipulation. First, we show that given a ciphertext generated by applying a substitution cipher to English text, the lexinvariant LM can be seen as implicitly approximating Bayesian inference of the lexical permutation, i.e., cipher key, in-context. To show this empirically, we train a small MLP probe on top of a frozen pretrained lexinvariant LM to predict the deciphered token corresponding to the last

[1] We specifically consider lexical symbols as tokens, not necessarily words or other linguistic units.

seen cipher token. We can then read out the inferred cipher key with each prefix of the sequence. We show that the accuracy of this inferred cipher key quickly improves as context length grows, reaching 99.6% average accuracy. We also show examples in Section 3.4 that visualize the uncertainties over different possible lexical mappings maintained by the lexinvariant LM when the cipher key is ambiguous and that the semantic meaning of a symbol with very rare occurrence can be inferred efficiently relative to other common symbols in context. Second, we show that lexinvariant models perform better than traditional models over synthetic pure in-context reasoning tasks that involve symbol manipulation. We observe a significant 4X improvement over a standard language model.

While the primary motivation of this paper is scientific exploration of a new idea, lexinvariance, we were also curious to see if it could help improve certain tasks, generalizing the performance gain we see on synthetic tasks. We stress that for most practical applications, lexinvariance is far too strong, so these experiments are intended to be illustrative rather than be a recipe for improving state-of-the-art. We discuss potential approaches to integrate the idea of lexinvariant LM into standard language modeling as a form of regularization, such that the LM assumes some form of partially stable symbol representations. The resulting LM can improve upon a standard language model over some BIG-bench tasks [23].

2 Lexinvariant Language Model

We define a language model as a probability distribution $p(x_1,\ldots,x_n)$ over input token sequences $x_1,\ldots,x_n \in \mathcal{V}^n$, where $\mathcal{V}$ is some vocabulary over symbols. A language model is lexinvariant if for all permutations $\pi:\mathcal{V}\to\mathcal{V}$ and for all token sequences $x_1,\ldots,x_n \in \mathcal{V}^n$, $p(x_1,\ldots,x_n)=p(\pi(x_1),\ldots,\pi(x_n))$. For example, if $\mathcal{V}=\{\text{a,b}\}$, then the model should assign the same probability to aab and bba. One example p that satisfies this could simply be

$$p(x)=\begin{cases}1/2 & x\in\{aab,bba\}\\ 0 & \text{otherwise}\end{cases} \tag{2}$$

Can such a lexinvariant language model predict language well, even though it can only make next token predictions based on the structure of co-occurence and repetition of input tokens in a single context?

2.1 Convergence on Language Modeling Performance

We show that we can construct a lexinvariant LM (as shown in Figure 2) to model the true language distribution faithfully, given a long enough context. The lexinvariant language model can essentially infer back the latent permutation π as it observes more symbols.

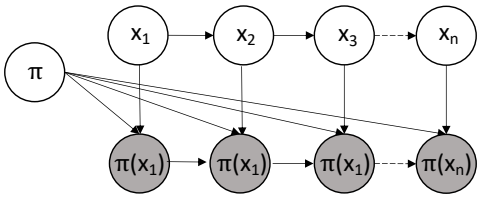


Figure 2: Probabilistic graphical model for the lexinvariant LM associated with the true language distribution $p(x_1,\ldots,x_n)$.

As an intuitive example, suppose that $\mathcal{V}=\{a,b\}$ and the true language only contains two sequences $babbbb$ and $ababab$ (and their prefixes) with even probability. When given only the first three letters, a lexinvariant model can't tell the latent permutation and can only assign the same probability to a and b for the next letter: Due to the lexinvariant property, it assigns the same probability to $p(a|aba)=p(b|bab)$ as well as to $p(b|aba)=p(a|bab)$. Further, $p(a|aba)=p(b|aba)$ because the permutations to prefixes aba and bab are equally probable. In contrast, when considering the prefix $abab$, the fourth letter resolves the ambiguity in possible permutations π. (Since $baba$ is not in the true language distribution, π cannot map a to b.) Therefore, the model can correctly predict that $p(a|abab)=1$ and $p(b|abab)=0$.

Formally, for a given language model p, we define the associated lexinvariant language model $p'(x_1,\ldots,x_n)$ as $\mathbb{E}_\pi[p(\pi^{-1}(x_1),\ldots,\pi^{-1}(x_n))]$. Analyzing it, we have the following theorem:

Theorem 2.1. *Let $x_1,\ldots,x_n$ be any token sequence generated by an arbitrary language distribution p with an alphabet of size d. Let $p'(x_1,\ldots,x_n)=\mathbb{E}_\pi[p(\pi^{-1}(x_1),\ldots,\pi^{-1}(x_n))]$. Then, for any*